

Experiment ‘p_6_4_sum_without_smallest_array’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_4_sum_without_smallest_array

Natural language statement: null

Method signature: p_6_4_sum_without_smallest_array(inputs: array<int>) returns (sum: int)

Ensures

- `sum == sum_array(inputs[..]) - find_smallest(inputs[..])`

Requires

- `inputs.Length >= 1`

Functional Code Given

```
function find_smallest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> ret <= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_smallest := find_smallest(inputs[1..]);
    var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    assert inputs == [inputs[0]] + inputs[1..];
    assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    ret
}

function sum_array(inputs: seq<int>): int
{
```

```
if |inputs| == 0 then 0
else inputs[0] + sum_array(inputs[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_6_4_sum_without_smallest_array(inputs: array<int>) returns (sum: int)`

The method should respect the following contract:

`requires inputs.Length >= 1, ensures sum == sum_array(inputs[..]) - find_smallest(inputs
↪ [..])`

The contract uses the following dafny code:

```
function find_smallest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> ret <= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_smallest := find_smallest(inputs[1..]);
    var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    assert inputs == [inputs[0]] + inputs[1..];
    assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    ret
}

function sum_array(inputs: seq<int>): int
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOHHEfpUYoWl0FNEA98eKg0CSH6i

Model: gpt-5.5-2026-04-23

Created at: 1780663767

Finish reason: stop

Usage: 692 tokens in, and 6880 tokens out

```
{
  var n := inputs.Length;
  var i := n - 1;
  var smallest := inputs[i];
  sum := inputs[i];

  assert i + 1 == n;
  var s0 := inputs[i..];
  assert |s0| == 1;
  assert s0[0] == inputs[i];
  assert s0[1..] == [];
  assert s0 == [s0[0]];
  assert s0 == [inputs[i]];
  assert sum_array(s0) == inputs[i];
  assert find_smallest(s0) == inputs[i];
  assert sum == sum_array(inputs[i..]);
  assert smallest == find_smallest(inputs[i..]);

  while i > 0
    invariant n == inputs.Length
    invariant 0 <= i < n
    invariant sum == sum_array(inputs[i..])
    invariant smallest == find_smallest(inputs[i..])
    {
      var restSum := sum;
      var restSmallest := smallest;
      i := i - 1;

      assert restSum == sum_array(inputs[i + 1..]);
      assert restSmallest == find_smallest(inputs[i + 1..]);
      assert 0 <= i;
      assert i + 1 < n;

      var s := inputs[i..];
      assert |s| >= 2;
      assert s[0] == inputs[i];
      assert s[1..] == inputs[i + 1..];
      assert s == [s[0]] + s[1..];
      assert s == [inputs[i]] + inputs[i + 1..];

      assert sum_array(s) == inputs[i] + sum_array(inputs[i + 1..]);
      assert find_smallest(s) == (if inputs[i] <= find_smallest(inputs[i + 1..]) then inputs
        ↪ [i] else find_smallest(inputs[i + 1..]));

      sum := inputs[i] + restSum;
      if inputs[i] <= restSmallest {
        smallest := inputs[i];
      }
    }
  }
}
```

```

    } else {
        smallest := restSmallest;
    }

    assert sum == inputs[i] + sum_array(inputs[i + 1..]);
    assert sum == sum_array(inputs[i..]);
    assert smallest == (if inputs[i] <= restSmallest then inputs[i] else restSmallest);
    assert smallest == (if inputs[i] <= find_smallest(inputs[i + 1..]) then inputs[i] else
        ↪ find_smallest(inputs[i + 1..]));
    assert smallest == find_smallest(inputs[i..]);
}

assert i == 0;
assert inputs[i..] == inputs[..];
assert sum == sum_array(inputs[..]);
assert smallest == find_smallest(inputs[..]);
sum := sum - smallest;
}

```

Final Program

```

function find_smallest(inputs: seq<int>): (ret: int)
    requires |inputs| >= 1
    ensures forall x :: x in inputs ==> ret <= x
{
    if |inputs| == 1 then inputs[0]
    else
        assert 2 <= |inputs|;
        var rest_smallest := find_smallest(inputs[1..]);
        var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
        assert inputs == [inputs[0]] + inputs[1..];
        assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
        ret
    }
}

function sum_array(inputs: seq<int>): int
{
    if |inputs| == 0 then 0
    else inputs[0] + sum_array(inputs[1..])
}

method p_6_4_sum_without_smallest_array(inputs: array<int>) returns (sum: int)
    requires inputs.Length >= 1
    ensures sum == sum_array(inputs[..]) - find_smallest(inputs[..])
{
    var n := inputs.Length;
    var i := n - 1;
    var smallest := inputs[i];
    sum := inputs[i];
}

```

```

assert i + 1 == n;
var s0 := inputs[i..];
assert |s0| == 1;
assert s0[0] == inputs[i];
assert s0[1..] == [];
assert s0 == [s0[0]];
assert s0 == [inputs[i]];
assert sum_array(s0) == inputs[i];
assert find_smallest(s0) == inputs[i];
assert sum == sum_array(inputs[i..]);
assert smallest == find_smallest(inputs[i..]);

while i > 0
  invariant n == inputs.Length
  invariant 0 <= i < n
  invariant sum == sum_array(inputs[i..])
  invariant smallest == find_smallest(inputs[i..])
{
  var restSum := sum;
  var restSmallest := smallest;
  i := i - 1;

  assert restSum == sum_array(inputs[i + 1..]);
  assert restSmallest == find_smallest(inputs[i + 1..]);
  assert 0 <= i;
  assert i + 1 < n;

  var s := inputs[i..];
  assert |s| >= 2;
  assert s[0] == inputs[i];
  assert s[1..] == inputs[i + 1..];
  assert s == [s[0]] + s[1..];
  assert s == [inputs[i]] + inputs[i + 1..];

  assert sum_array(s) == inputs[i] + sum_array(inputs[i + 1..]);
  assert find_smallest(s) == (if inputs[i] <= find_smallest(inputs[i + 1..]) then inputs
    ↪ [i] else find_smallest(inputs[i + 1..]));

  sum := inputs[i] + restSum;
  if inputs[i] <= restSmallest {
    smallest := inputs[i];
  } else {
    smallest := restSmallest;
  }

  assert sum == inputs[i] + sum_array(inputs[i + 1..]);
  assert sum == sum_array(inputs[i..]);
  assert smallest == (if inputs[i] <= restSmallest then inputs[i] else restSmallest);
  assert smallest == (if inputs[i] <= find_smallest(inputs[i + 1..]) then inputs[i] else
    ↪ find_smallest(inputs[i + 1..]));
  assert smallest == find_smallest(inputs[i..]);
}

```

```
}  
  
assert i == 0;  
assert inputs[i..] == inputs[..];  
assert sum == sum_array(inputs[..]);  
assert smallest == find_smallest(inputs[..]);  
sum := sum - smallest;  
}
```

Total Token Usage

Input tokens: 692

Output tokens: 6880

Reasoning tokens: 6135

Sum of 'total tokens': 7572

Experiment Timings

Overall Experiment started at 1780663767258, ended at 1780663936419, lasting 169161ms (169.16 seconds)

Iteration #1 started at 1780663767259, ended at 1780663936419, lasting 169160ms (169.16 seconds)